

Trading Book PnL — FIFO Engine

Incremental Realised & Unrealised P&L — Model Documentation

Risk Modelling — F. Balloni

May 2026

1 Overview

The Trading Book PnL engine computes daily **realised** and **unrealised P&L** for the fractional-share trading book using a strict **First-In, First-Out (FIFO)** lot-matching methodology. The calculation runs incrementally: it picks up from the last cached open-position snapshot in Snowflake, processes all new trades up to the previous business day, and writes refreshed position and cache parquet files to S3.

Entry point: `automation/task_pnl_trading_book.py`

Core engine: `risk_models/pnl_fifo.py` — `initiate_pnl_engine`, `pnl_realised_polars`

2 Methodology

2.1 Sign Convention

Each trade carries a signed quantity determined by its direction:

$$q_i = \begin{cases} +|q_i| & \text{side} = \text{B} \quad (\text{buy}) \\ -|q_i| & \text{side} = \text{S} \quad (\text{sell}) \end{cases} \quad (1)$$

2.2 FIFO Realised P&L

Two queues are maintained per symbol: `fifo_queue` (unmatched buy lots) and `short_queue` (unmatched sell lots). Each lot preserves its individual execution price.

Sell trade — matched against the oldest buy lots:

$$\text{rPnL}_{ik} = m_{ik} \cdot (p_i - p_k^B), \quad m_{ik} = \min(|q_i^{\text{rem}}|, q_k^B) \quad (2)$$

Buy trade (short cover) — matched against the oldest short lots:

$$\text{rPnL}_{ij} = m_{ij} \cdot (p_j^S - p_i), \quad m_{ij} = \min(q_i^{\text{rem}}, q_j^S) \quad (3)$$

Total realised P&L per trade:

$$\text{rPnL}_i = \underbrace{\sum_j m_{ij}(p_j^S - p_i)}_{\text{short cover}} + \underbrace{\sum_k m_{ik}(p_i - p_k^B)}_{\text{long book}} \quad (4)$$

2.3 Unrealised P&L

Residual open lots are marked to end-of-day prices:

$$\text{uPnL}_s^{(t)} = \sum_{\ell \in \text{open}(s)} (P_t^{\text{EOD}} - p_\ell) \cdot q_\ell^{\text{open}} \quad (5)$$

2.4 P&L Identity

$$\text{Total PnL}^{(T)} = \sum_i \text{rPnL}_i + \text{uPnL}^{(T)} \quad (6)$$

2.5 Incremental Day Loop

The engine iterates over weekdays $d \in [t_{\text{start}}, t_{\text{end}}]$:

$$t_{\text{start}} = \max(\text{cache report_date}) + 1, \quad t_{\text{end}} = \text{today} - 1 \quad (7)$$

At each step the residual open lots from day $d-1$ are prepended to the new trades before matching: $\mathcal{T}^{(d)} = \text{cache}^{(d-1)} \oplus \text{trades}^{(d)}$

2.6 Corporate Action Handling

Instruments with `booking_category = 'CORPORATE_ACTION'` are routed to `pnl_realised_ca`, which adjusts historical quantities by the relevant split ratio before FIFO matching. All other instruments are processed by `pnl_realised_polars` (Polars-accelerated).

3 Parameters

Parameter	Type	Example	Description
<code>acct</code>	int	9800001301	Account number
<code>tr_sd</code>	date	<code>date(2026,1,2)</code>	Trade window start
<code>tr_ed</code>	date	<code>date(2026,4,30)</code>	Trade window end
<code>ca_sd</code>	date	<code>date(2026,1,1)</code>	Cache snapshot date
<code>refetch_cache</code>	int	1	1 = re-query Snowflake
<code>save_data</code>	int	1	1 = write parquet at end
<code>save_daily</code>	int	0	1 = write per-day parquet
<code>data_src</code>	str	<code>'share_booking'</code>	Trade data source
<code>trade_qry</code>	str	<code>'old'</code>	Query flavour (<code>old/new</code>)
<code>pth</code>	str	<code>'/tmp'</code>	Local output path
<code>verbose</code>	bool	False	Print progress

Accounts and data sources:

Account	data_src	trade_qry
9800001301	share_booking	old
9800003301	share_booking	old
9800001601	inquisitor	new
9800003601	inquisitor	new

4 Outputs

Key	Type	Content
<code>out_pos</code>	DataFrame	Daily positions, rPnL, uPnL per symbol
<code>out_cache</code>	DataFrame	Open FIFO lots for next-day carry-forward
<code>out_trade_pnl</code>	DataFrame	Trade-level realised P&L
<code>fname_pnl</code>	str	Path to position parquet (S3 upload target)
<code>fname_cache</code>	str	Path to cache parquet (S3 upload target)

5 Advantages and Limitations

Advantages	Limitations
Exact lot-level attribution	Python loop; not vectorised over symbols
Preserves individual short prices	Memory-intensive for large trade histories
Incremental — only processes new days	Requires reliable EOD price series
Handles corporate actions separately	Missing prices backfilled to 1 (silent risk)
Audit-ready cache chain	Three ISINs hard-excluded from feed

6 Usage Example

```
from risk_models import pnl_fifo
from datetime import date

out = pnl_fifo.initiate_pnl_engine(
    acct          = 9800001601,
    tr_sd         = date(2026, 1, 2),
    tr_ed         = date(2026, 4, 30),
    ca_sd         = date(2026, 1, 1),
    refetch_cache = 1,
    save_data     = 1,
    save_daily    = 0,
    data_src      = 'inquisitor',
    trade_qry     = 'new',
    pth           = '/tmp',
    verbose       = False,
)

out['out_pos']      # daily positions + PnL
out['out_cache']    # open lots to carry forward
out['out_trade_pnl'] # trade-level rPnL
```